Lecture. — 02.

11 July 2026.
Saturday.

Logic → rational.

Rational → maximum goals achieve করার জন্য করে। through measure করে utility।

Identify the AI -types:

Recommending a movie → Rational.

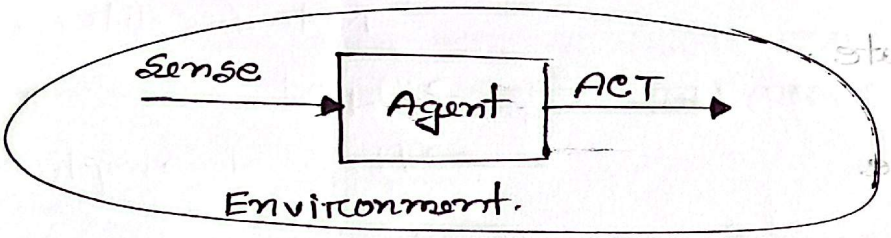
AI → Rational
AI → Humanly.

Important: ***

Topic: Agents and Environments.

Agent should be anything যার capability আছে information sense করা environment থেকে এবং সেই অনুযায়ী act করা।

for ex: Human, Robots, software etc.



percept: instant যেই information টা agent পায়।

percept sequence: From the beginning to the End যেই information পায়ি। (পুরো History আরকি).

→ এর উপর base করে যেই action নিবো সেটা Agent function

Agent এর access যেই পর্যন্ত আছে সেই পর্যন্ত environment।

Omniscience : perfect। কোনো errors থাকবে না।

Specifying the task environment ⟶ *** important for exam

→ PEAS (Performance, Environment, Actuators, Sensors).
output organs, input

Lecture - 03

14 July 2026 (Tuesday)

Search problem :-

i) State space

ii) Initial state.

iii) Goal state

iv) Actions.

v) Transition Model (কোনগুলো একটা actions এর পর কোন position এ থাকবে)

vi) Path cost.



properties of task environments.

Types of environment

Fully observable : যেমন একটা agent যে at a time সবকিছু দেখতে পারে।
যেমন : Chess Board.

partially observable: at a time সবকিছু দেখতে পারে না। যেমন : self driving car.

environments যে interaction এর দরকার নাই, একাই।

Single agent : একাই কাজ করে। যেমন : Vacuum cleaner.
আর কোনো agents নেই।

Multiple agent : chess playing Robots, self driving cars

Deterministic :
আমি এখন যা করবো তার উপর depend করে next state হবে। যেমন : chess playing robots.

Stochastic :
environment এর change শুধু - আমার actions এর উপর depend করে না। যেমন : Taxi driving

Random change
agent এর control নেই।

Episodic :
current actions যদি future actions এ effect না করে।

Sequential : Current actions, future actions কে effect করবে। যেমন : chess, Automated taxi driving.

CamScanner

ix). Daynamic :
Agents এর actions ছাড়াই, changes আসে environ-ments এ।

x) Static : Agents এর actions এর উপর depend করে।

xi) Discrete : Finite number of states finite.
যেমন : Chess.

xii) Continious : জানি না কতগুলো states possible.
ex : Taxi driving.

Note (এই slide থেকে এগুলো পড়বো)

→ Agents & environments concept
→ PEAS descriptions.
→ Search problem
→ types of environments.

18 July 2026. Lecture - 01. (Saturday).

Uninformed search

→ goal সম্পর্কে কোনো idea নেই।

BFS
DFS — LIFO
UCS
DL
IDS
BS

✓ Tree তে multiple path বা structure বার বার আসবে।
✓ Graph এ multiple আসবে না।

Fringe : List এর মধ্যে কোন কোন path visit হয়েছে and কোন কোন node বাকি।

Expansion : যেই node এ আছি তার সব child visit করে ফেলেছি সেই node টা expanded.

Exploration strategy :

DFS

Time complexity = $O(b^m)$

Space complexity = $O(bxm)$ → at a time কতটুকু space লাগে

optimal solution দিবে না।

BFS (level by level)

i) complete ? → yes.

ii) optimal solution ? → দিবে (যদি সব edge এর value গুলো same হয়).

iii) Time complexity = $O(b^s)$; $s = 1, 2, \cdots m$
↳ worst case এ কতগুলো node explore করা লাগে

iv) Space complexity = $O(b^s)$

CamScanner

✓ DFS VS BFS.

Depth Limited Search.

DFS এর মতই খুব limited থাকবে extra.

complete ? → level এর পর goal exit করলে complete হবে না।

Time complexity : $O(b^l)$

space complexity : $O(b \times l)$

Iterative Deepening Search.

BFS and DFS algorithm এর advantage নিতে চাইলে এইটা use করবো। Search করবে DFS দিয়ে।

I0 : প্রথম Depth (0) তে soln আছে ? → না। তাই level বাড়াবে।

I1 : . . . . .

complete ? → না ও হতে পারে। হতেও পারে। soln পেলে bottom soln ই পাবো।

Uniform Cost Search.

→ cost wise search করবে।
→ যেই path এ তুলনে cost কম তেটা considers করবে।

complete ?

optimal ? → Yes. cause lowest cost এ solution দিয়ে।

time complexity : $O(b^{C^*/\varepsilon})$

space complexity : $O(b^{C^*/\varepsilon})$

C^* → total cost
ε → edge cost.

$\frac{C^*}{\varepsilon}$ → depth.

branching → যতগুলো child possible

21/07/2026 (Tuesday)
Lecture – 05

CT 01 (Next tuesday)

Uninformed search → BFS, DFS, DLS, IDS, UCS, BS.
time complexity, কোনটা কখন better, এদের properties

Bidirectional Search (FIFO)

start and goal থেকে at a time search হবে। এবং এদের intersection থাকবে মানে common একটা node থাকবে।
→ main target হল finding the path efficiently.
→ goal state অবশ্যই জানা থাকা লাগবে।

BFS vs BS

Forward

Visited

1
4
2
8
9
6
8
10

Expand

(কোন node এর সব child যদি visit করা হয়ে যায়)

1
4
2
8
9

1 → 4 → 8 → 9

Backward

V

16
12
10
15
9
11
8

E

16
12
10
15
9

16 → 12 → 10 → 9

∴ 1 → 4 → 8 → 9
16 → 12 → 10 → 9

∴ 1 → 4 → 8 → 9 → 10 → 12 → 16

BFS

$O(b^s)$; s = solution এর depth

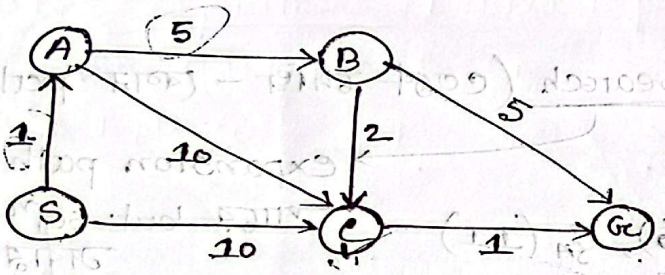
BS

$O(b^{\frac{s}{2}} + b^{\frac{s}{2}})$

$= O(2b^{\frac{s}{2}})$

$= O(b^{\frac{s}{2}})$ → ~~both~~ for time and space.

Example



Find out:

order of expansion
solution paths
paths costs.

apply — DFS, BFS, UCS (starting node এর S) destination — Gc

Soln: 1) BFS : (FIFO)

BFS এ প্রথমেই solution আসবে না

visited	expansion
~~S~~	S
~~A~~ S	A
~~C~~ S	C
~~B~~ A	B
~~Gc~~ C	Gc

∴ order of expansion

$S \rightarrow A \rightarrow C \rightarrow B \rightarrow Gc$

Solution paths:

$S \rightarrow C \rightarrow Gc$

path cost:

$10 + 1 = 11$

Gc টা প্রথম আসছে C থেকে।

A, e A

② DFS (LIFO)

DFS এ solution পরবর্তিক- হতে পারে।

V	E
S	S
A, C	C
A, Gc	Gc

order of Expansion : S → C → Gc
solution paths : S → C → Gc
costs : 11.

③ uniformed cost search (cost বেশি – কোন path এ cost কম)

S: A(1), C(10)

expansion path change হতে পারে but soln same-ই হবে সবার।

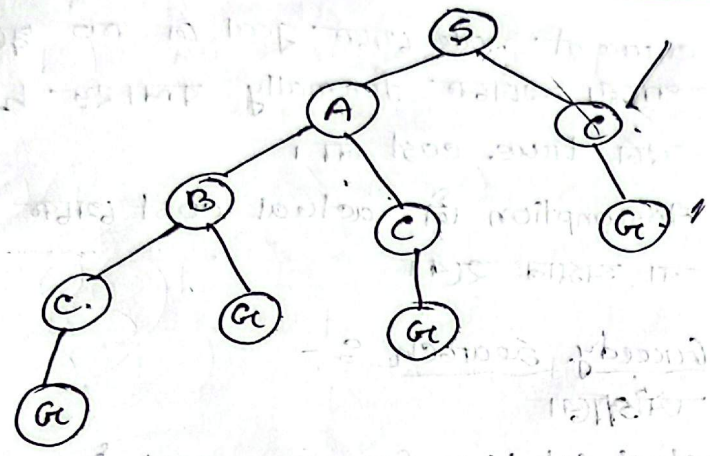
A(1): C(10), B(6), C(11)

B(6): C(10), C(11), C(8), Gc(11)

C(8): C(10), C(11), Gc(11), Gc(9)

Gc(9)

S → A → B → C → Gc



BFS (FIFO)

V	E
~~A~~	A
~~Cs~~	Cs
~~BSA~~	BSA
~~CSA~~	CSA
~~GSC~~	GSC
CSAB	
GSAB	
GSAC	

$S \rightarrow C \rightarrow G$

Lecture – 06.

Topic : Informed Search.

Assumption / estimate : current node থেকে goal টা কত দূরে থাকতে পারে। এটাকে formally বলা হয় heuristic (h)
এটা true cost না।
Assumption টা actual cost থেকে কম হবে বা সমান হবে।

Greedy Best ~~Fast~~ First search / Greedy search :-

↳ heuristic (h) value ধরে এগোনো
↳ খারাপ হলে actual solution দিবে না, goal টা wrong হতে পারে।
↳ ভালো।। কম node expend করে goal এ পৌঁছাতে পারবো।

A* Search :

$$f = (g+h)$$

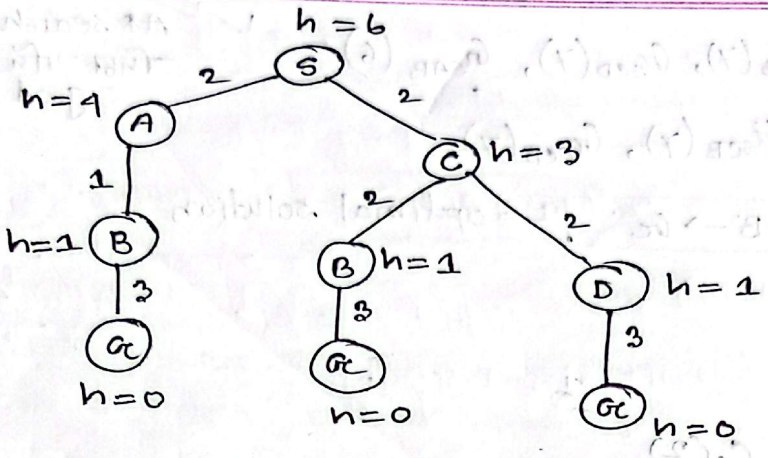
(h) value প্রত্যেকের জন্য individual.
যেই node এ আছি ঐ node থেকে ই (h) এর value.
(g) value হল আগের node থেকে এখন যেই node এ আছি তাদের যোগফল।

~~A*~~ tree and A* graph diff.

UCS → truth path cost (g)

Greedy BFS → heuristic value (h)

Practice



i) Greedy search :- (consider only 'h' value)

S : A(4), C(3)

$C_S(3)$: A(4), $B_{SC}(1)$, $D_{SC}(1)$

expansion

$B_{SC}(1)$: A(4), $D_{SC}(1)$, $G_{SCB}(0)$

S → C → B → Gc

cost = 2+2+3 = 7.

(ii) A* tree : (consider (g+h) value → source cities total, → specific)

S : A(6), C(5)

$C_S(5)$: A(6), $B_{SC}(5)$, $D_{SC}(5)$

$B_{SC}(5)$: A(6), $D_{SC}(5)$, $G_{SCB}(7)$

$D_{SC}(5)$: A(5), $G_{SCB}(7)$, $G_{SCD}(7)$

A(6) : $G_{SCB}(7)$, $G_{SCD}(7)$, $B_{SA}(4)$

$B_{SA}(4)$: $G_{SCB}(7)$, $G_{SCD}(7)$, $G_{SAB}(6)$

A* search optimal solution পাওয়া যদি heuristic value good হয়।

$G_{SAB}(6)$ ~~$G_{SCB}(7)$, $G_{SCD}(7)$~~

$\therefore$ $S \to A \to B \to G$ → optimal solution.

Cost : 6.

UCS :

S : $A_S(2)$, $C_S(2)$

$A_S(2)$: $C_S(2)$, $B_{SA}(3)$

$C_S(2)$: $B_{SA}(3)$, $B_{SC}(4)$, $D_{SC}(6.4)$

$B_{SA}(3)$: $B_{SC}(4)$, $D_{SC}(4)$, $G_{SAB}(6)$

$B_{SC}(4)$: $D_{SC}(4)$, $G_{SAB}(6)$, $G_{SCB}(7)$

$D_{SC}(4)$: $G_{SAB}(6)$, $G_{SCB}(7)$, $G_{BCD}(7)$,

$G_{SAB}(6)$

$\therefore$ $S \to A \to B \to G$ → optimal solution.

Cost → 6.

Heuristic value যত ভালো হবে node expension তত কম হবে।

Lecture 3y.
28-07-2026

Admissible Heuristics.

→ যেই heuristic optimal way তে goal এর দিকে নিয়ে যায়। ☺

$$0 \leq h(n) \leq h^*(n)$$

$h^*(n)$ → true cost

যদি সবগুলো heuristic value admissible heuristics value হয় তাহলে এ graph optimal solution দিবে।

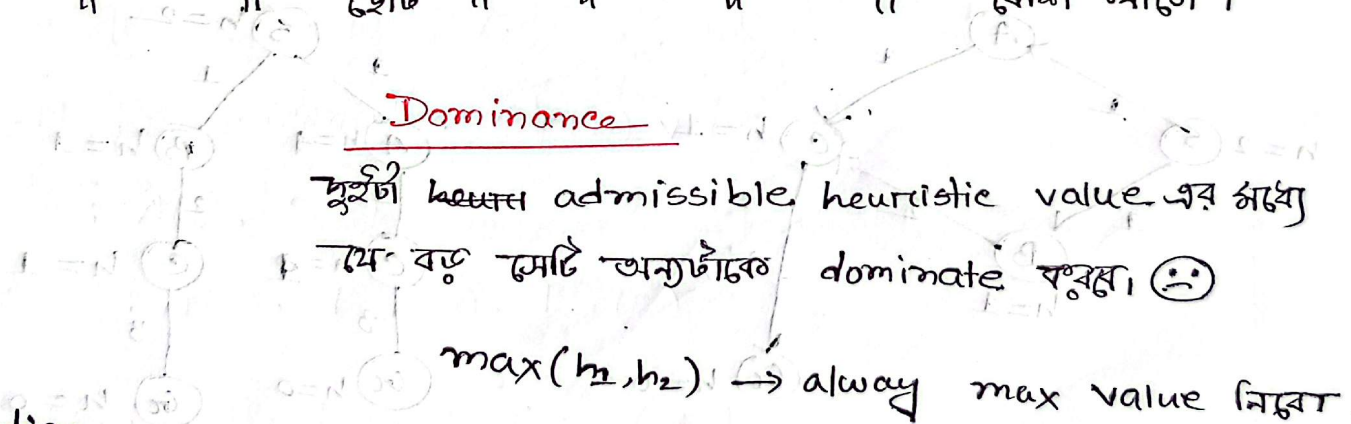
** যেকোনো problem এর relaxed/easy version এ যেই cost পাবো সেটাই হবে admissible heuristics.

~~দুইটা admi~~
~~** heuristic value optimal solution এর কাছাকাছি~~

** দুইটা admissible heuristic value এর মধ্যে বড় value টা more accurate বা better কারন এটা true cost এর বেশি কাছাকাছি।

Admissible heuristic value বেশি এর জন্য node expansion কম লাগে।

" " " ছোট " " " বেশি লাগে।



Dominance

দুইটা ~~heuris~~ admissible heuristic value এর মধ্যে যে বড় সেটি অন্যটাকে dominate করবে। ☹

$\max(h_1, h_2)$ → always max value নিবো।

practice

h_5, h_8 → inadmissible.

h_9 → is the best.

CamScanner

Lecture – 08.
01/08/2026.

Slide

(b) $\underset{\text{admissible}}{h_1}, h_2, \underset{\times}{h_3}$

i) $h_4(n) = \max(h_1, h_2, h_3)$ → inadmissible.

ii) $\text{Max}(h_1, h_2)$ → h_3 admissible যেহেতু $0 \le h(n) \le h^*(n)$ পর্যন্ত ছাড়িয়ে যাবে।

i) $h_5(n)$ = possibly inadmissible.

ii) $h_6(n)$ = in

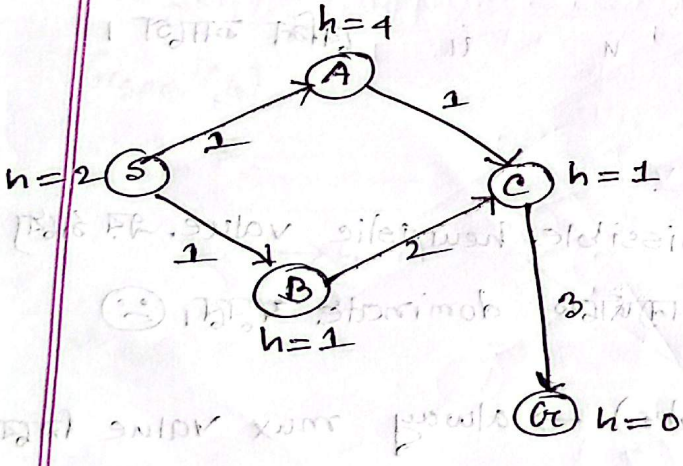
iii) h_7 = inadmissible.

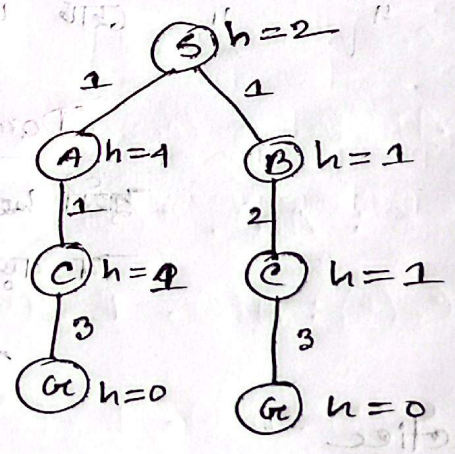
iv) h_8 = ~~possibl~~ inadmissible.

v) $h_9(n)$ = inadmissible.

Graph Search

একটা node একবারের বেশি expand করবো না।





$f = g + h$

S : $A_S(5)$, $B_S(2)$

$B_S(2)$: $A_S(5)$, $C_{SB}(4)$

$C_{SB}(4)$: $A_S(5)$, $G_{SBC}(6)$

$A_S(5)$: $G_{SBC}(6)$, $C_{SA}(3)$ ✗ → এখান থেকে C কে নিবো না কারণ C আগে expand করে ফেলেছি।

$S \rightarrow B \rightarrow C \rightarrow G$

এই problem এ A* tree search optimal solution দিচ্ছে কিন্তু এই same problem A* tree graph search optimal solution দিচ্ছে না।

Consistency of Heuristics.

কোনো দুইটা node এর heuristics value এর difference যদি ঐ node গুলোর actual cost এর সমান / ছোট হয় তাহলে heuristic consistence.

$$h_A - h_B \leq \text{True path cost from } A \rightarrow B.$$

A* Graph Search

A* Graph search optimal solution দিবে যদি heuristic value admissible and consistence হয়।

A* tree Search

- optimal solution দিবে যদি heuristic value admissible হয়।

Question : কোনো একটা graph দিয়ে বলতে পারবে A* search এ graph নাকি

tree optimal solution দিবে?

* inadmissible হলে consistence হবে না।
* UCS এ $h = 0$
* Greedy তে শুধু h value নিয়ে কাজ করে
* tree search এ $f = h + g$.

DFS apply. Memory Bounded informed search.

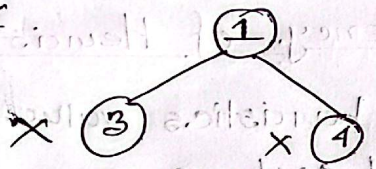
IDA* Algorithm: $f = g + h$ এখানে যেটা নিয়ে কাজ করে,

threshold = 1 এর মধ্যে যেই যেই node আছে তাদের f value এর সাথে মিলাবো।

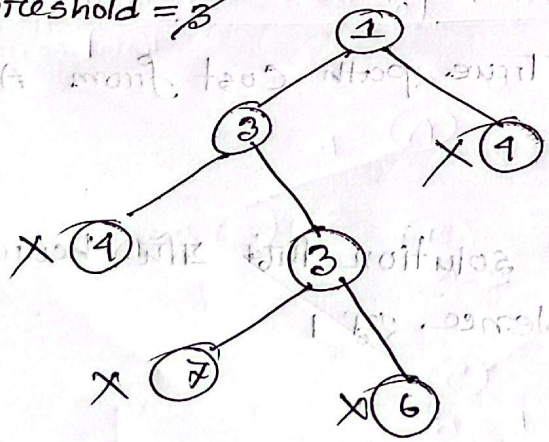
1st

threshold = ~~x~~

goal = 14



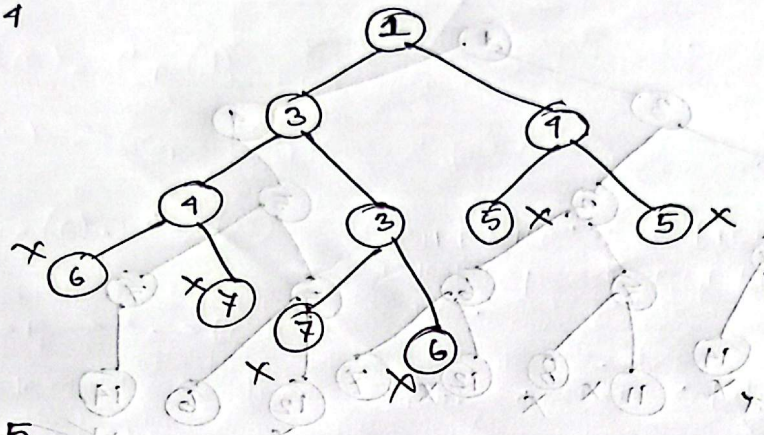
2nd threshold = ~~3~~



fail node এর মধ্যে minimum নিবো।

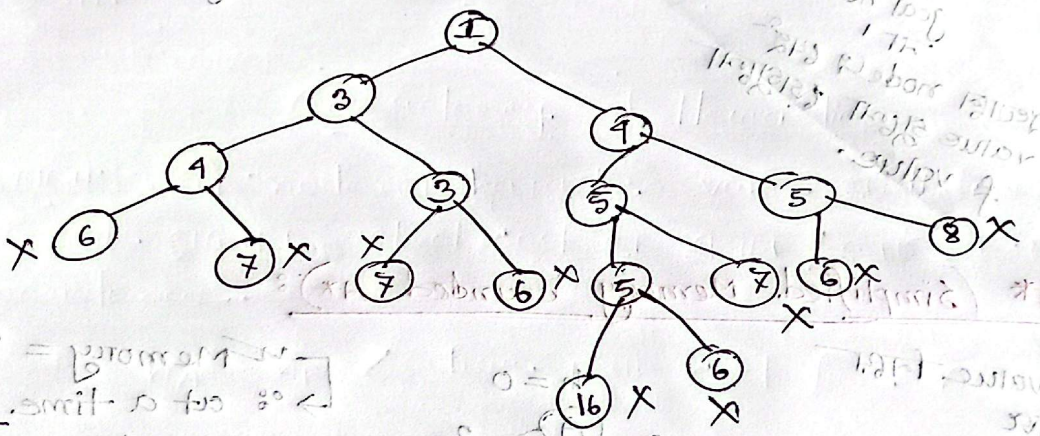
3rd

Threshold 4



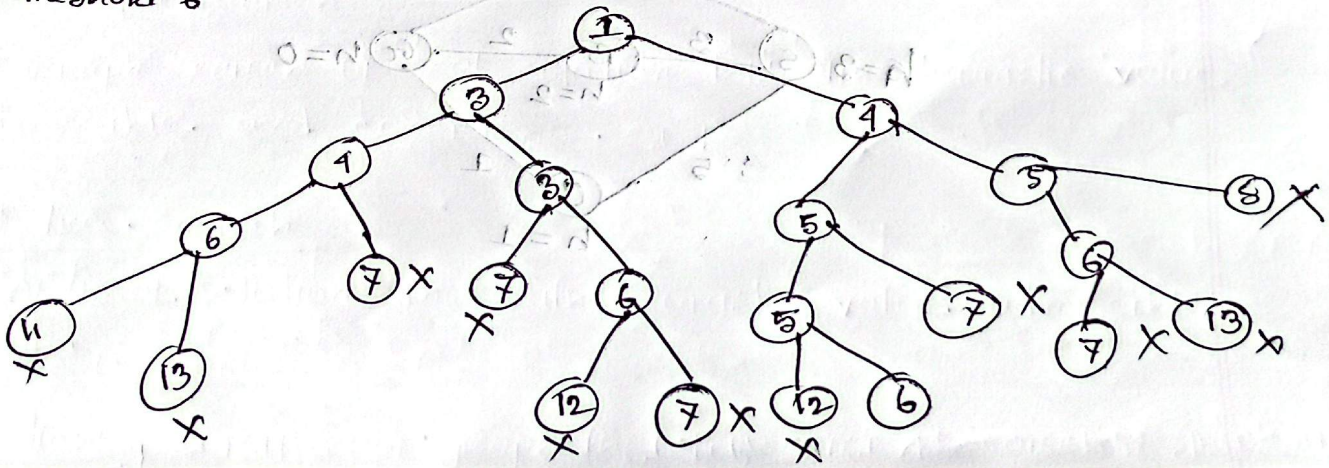
4th:

Threshold 5

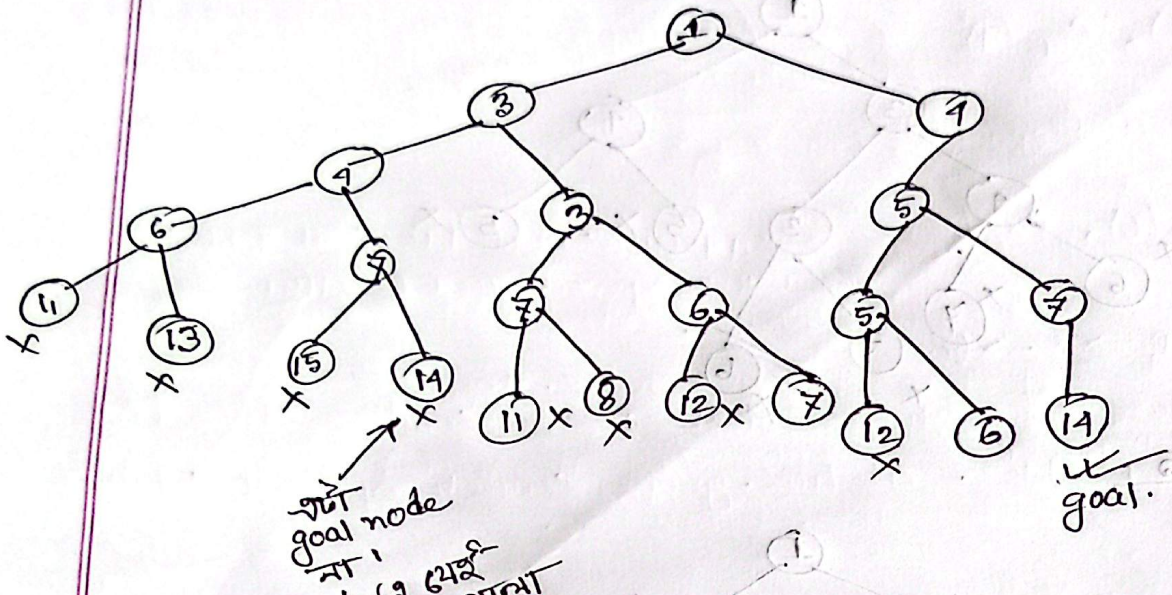


5th:

Threshold 6



threshold 7



এটা goal node না।

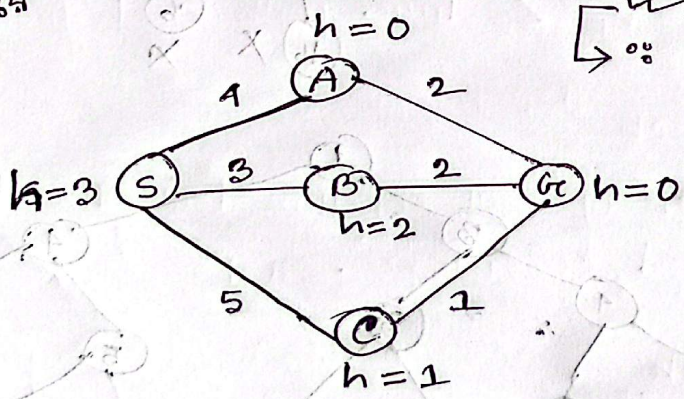
এখানে node এর value গুলো হচ্ছে f value.

goal.

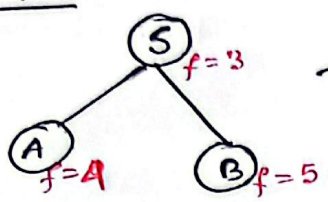
SMA* (Simplified Memory bounded A*) :-

f value নিয়ে কাজ

Memory = 3 nodes.
∵ at a time 3 টা node রাখতে পারবো

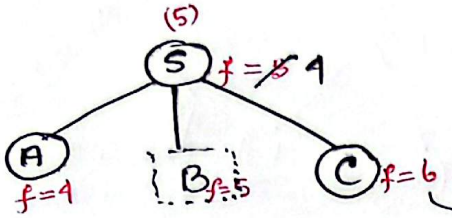


Step 1



→ 3 টা node আছে। এখন অন্য node কে expand করার জন্য ৮r child এর f value বেশি তাকে সরিয়ে দিবো

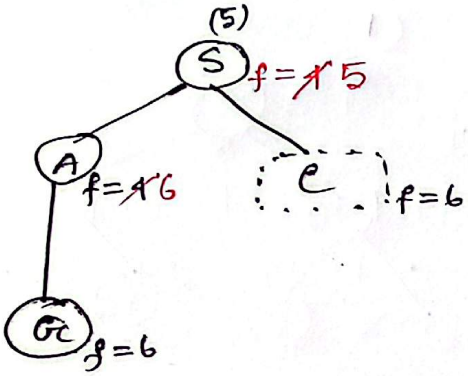
Step 2



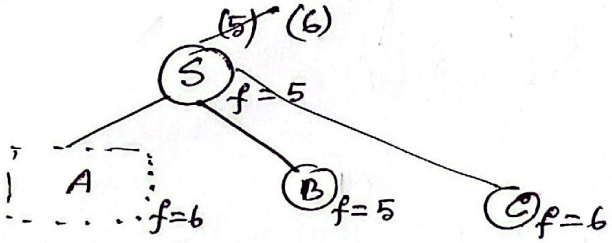
যখন একটা node এর সবগুলো child explore করে ফেলবো তার মধ্যে যে best cost সেটা নিয়ে parent টাকে update করবো।

→ store করা value টা C এর f value থেকে ভালো তাই আর store করবো না।

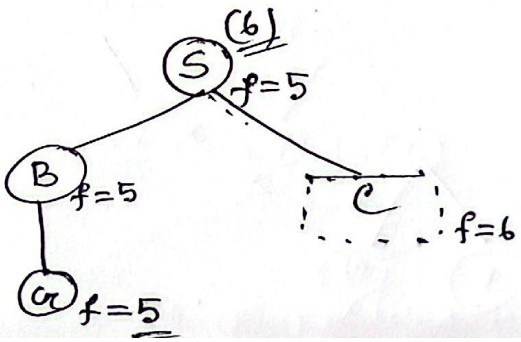
Step 3



Step 4



Step 5



5<6

goal : S → B → Gc

4/08/2026
Tuesday

Recursive best first search.

Rules

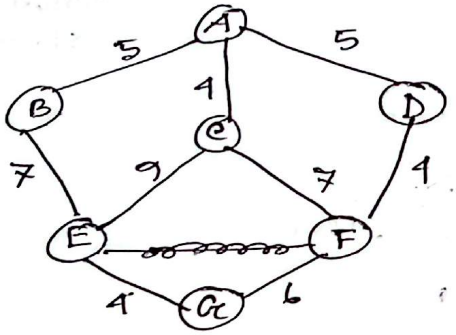
1) $s.f = \max(s.g + s.h, node.f)$ → নিজের যেই f value তার সাথে parent এর f value compare করে তার থেকে max নিবো।

2) Best successor → যার f value কম।

3) যেকোনো node কে তখনই explore করবো যখন ঐ node এর f value, f_limit থেকে ছোট/সমান হয়। যদি Best.f > f_limit তাহলে explore করবো না।

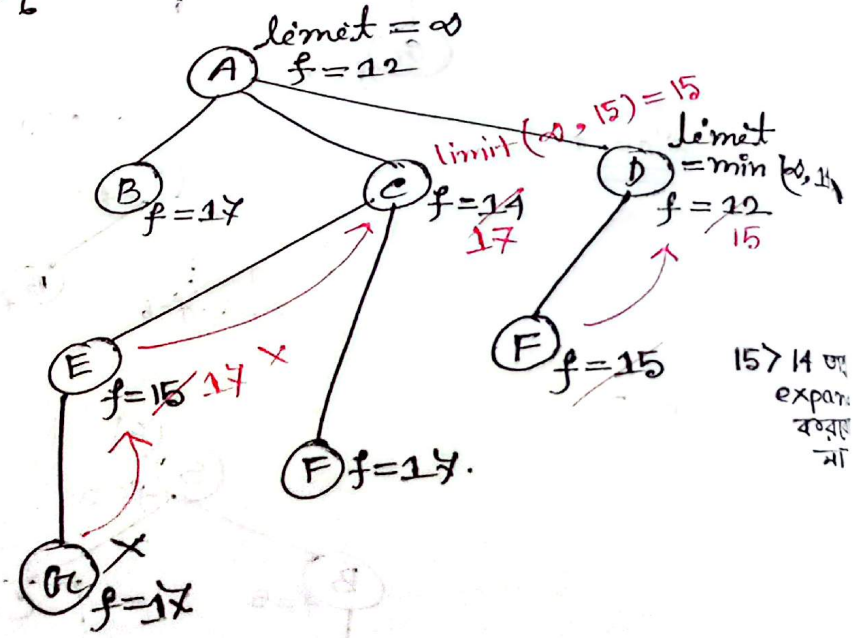
4) $f_limit = \min(f_limit, alt.f)$ → সেটাকে expand করতেছি তার child এর।

A start node
Gc Goal node.



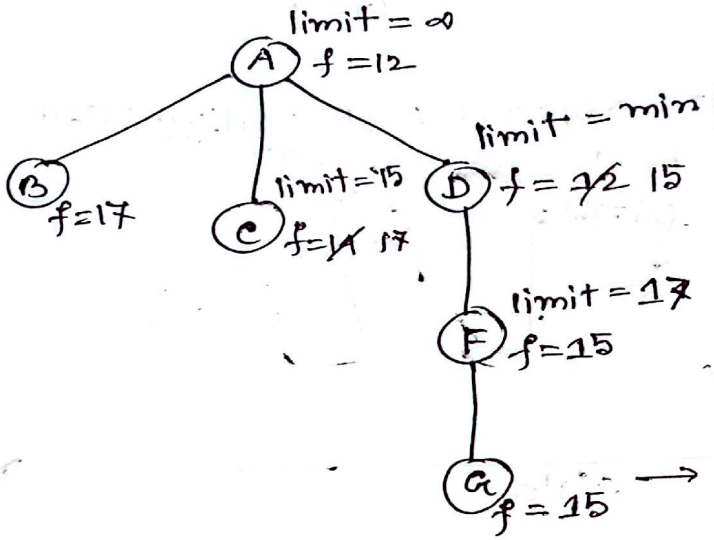
শুরুতে start node কে limit = ∞ দিবো

A	12
B	12
C	10
D	∞
E	2
F	6
Gc	0



যখন কোনো node fail করবে তখন তার parent কে ঐ node এর f value দিয়ে replace করবো।

expand করতে চাইলে তাকে আগে limit দেয়া লাগবে।
goal এ পৌঁছালেও দেখতে হবে rules 3। যদি fail করে তাহলে Back এ যাওয়া লাগবে।



→ rules ③ fail করেনি। তাই solution পেয়েছি।

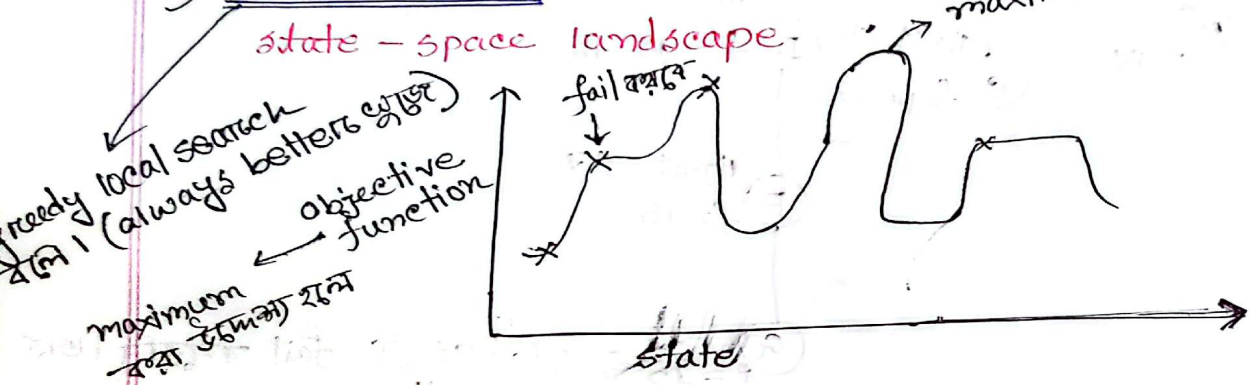
A → D → F → G

(Local Search) → যেকোনো way তে goal এ যাওয়াই main target.

- current path টা শুধু মনে রাখে।
- memory কম লাগে।
- সবগে আটকে যাওয়ার problem নেই।

i) **Hill-Climbing:** → সবসময় best solution পাবো না।

state-space landscape.



~~Example: 8-queens problem~~.

(*) Hill climbing কোথায় stuck হয়।

↓

local maxima, Ridges, plate--→ shoulders → theory আসতে পারে।

Hill-climbing search → conceptual আছে।

Variants

i) Steepest-ascent version [out of all the state → maximum এর মধ্যে best নিবে]

ii) Stochastic hill climbing → Randomly choose করবে।

iii) First-choice hill climbing → একটা একটা করে node দেখবে যে improve আছে নিবে। parent থেকে improve যার হয়েছে। প্রথম improve টা-কেই নিবে। পরে কি আছে, improve হবে কিনা দেখবে না।

iv) Random-restart hill climbing : যেই জায়গা তে stuck হবে তারপর randomly যেকোনো জায়গা থেকে আবার restart করবে।

8/08/2026 (Saturday)

neighboring node থেকে improvement নিবে।

Simulated annealing :- $e^{\frac{\Delta E}{T}}$, randomness allow করে এই $e^{\Delta E/T}$ probability নিয়ে।

ΔE = দুইটা state এর মধ্যে badness, current state and next state এর value এর মধ্যে minimum চাইলে min নিবে। max চাইলে max নিবে।

T = current node টাকে কতটুকু flexibility দিচ্ছি। initially T এর value high থেকে start করি।

$$e^{\Delta E/T} \doteq [0,1]$$

↳ compare করবো threshold এর সাথে।

* Direct improvement পেলে $e^{\Delta E/T}$ বের করা লাগবে না।

negative মানে improvement না।

$\Delta E = E(\text{neighbor}) - E(\text{current})$ ← Question ↓ এ বলা আছে।

(1)

Maximization problem

$T = 8 - 2 * t$ → দেওয়া থাকবে।

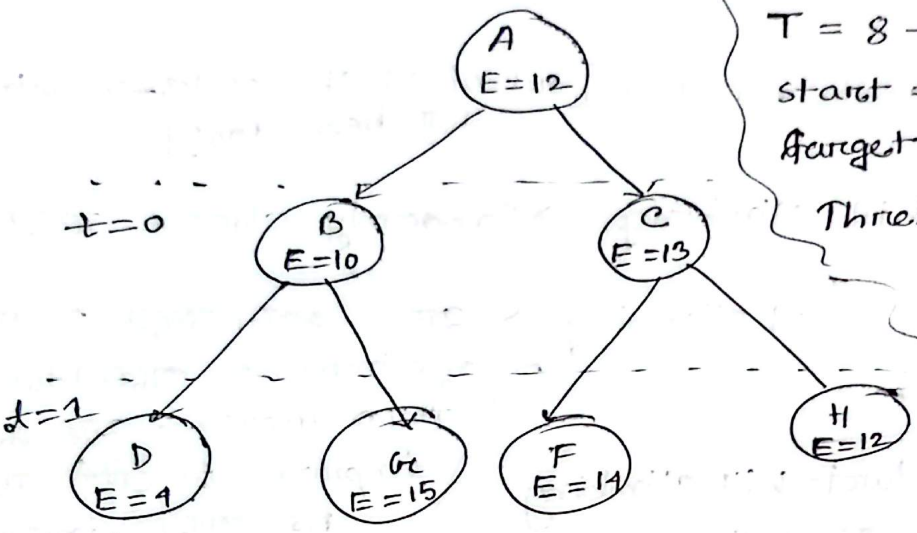
start = A

target = G

Threshold → 0.5

↳ সমান বা max হলে accept করবো।

left to Right যাবো



i) Hill climbing এ F এ যাবে। F আমাদের goal না (২/৩)।

ii) For B,

$T = 8 - 2 \times 0 = 8$

$e^{\Delta E/T} = e^{-2/8} = 0.78 > 0.5$ (accept করবো)

For D, → value কমাতেছে তাই $e^{\Delta E/T}$ বের করবো,

$t = 8 - 2 \times 1 = 6$

$e^{\Delta E/T} = e^{-6/6} = 0.37 < 0.5$ (accept করবো না)

For G, $\Delta E = +5$ ↳

max [বসানো neg]
min [বাড়ানো neg]

$T = 10 - 3t$.

minimization problem এটা,

For A,

$e^{\Delta E/T} = e^{-5/10} = 0.61 < 0.88$ $t = 0$, $T = 10 - 3\times 0 = 10$

For B, $e^{\Delta E/T} = e^{-10/7} = 0.24 < 0.67$ $t = 1$, $T = 10 - 3 = 7$

$t = 2$

For C, $\Delta E = +$ ve

Home-work → figure 4(a). → maximization problem

Soln:

For P,

$\Delta E = -5$, $e^{-5/67} = 0.93$, RNG = 0.99 ✗ ; $t = 0$

For Q,

$\Delta E = -9$, $e^{-9/66} = 0.87$, RNG = 0.990 ✗ ; $t = 1$

For R,

$\Delta E = -2$, $e^{-2/61} = 0.97$, RNG = 0.98 ✗ ; $t = 2$

For S,

$\Delta E = -13$, $e^{-13/52} = 0.78$, RNG = 0.97 ✗ ; $t = 3$

For T,

$\Delta E = +3$ ✓ ; $t = 4$.

11/08/2026.
Tuesday.

CT 02
Next Saturday
Syllabus : Informed search (simulation + conceptual).
(till → IDA**)

Topic : Beam Search.

Hill climbing এ একটা node নিয়ে চলে।

multiple state নিয়ে কাজ শুরু করে।

Local beam search : Best k choose করে neighboring থেকে।

Q. Hill climbing and beam search এর difference.

Topic : Genetic Algorithm.

Fitness function → population এর value calculated বলবো।

Example : $f(x) = (a+b) - (c+d) + (e+f) - (g+h)$

(a)

$x_1 = 65413532$; $f(x_1) = 9$ ✓

$x_2 = 87126601$; $f(x_2) = 23$ ✓

$x_3 = 23921285$; $f(x_3) = -11$

$x_4 = 11852094$; $f(x_4) = -19$

(best দুইটা নিবো)
↓
question এ বলে দিবে।

(b)

6 5 4 1	3 5 3 2
8 7 1 2	6 6 0 1

↓

6 5 4 1	6 6 0 1
8 7 1 2	3 5 3 2

CamScanner

Topic : Adversarial Search.



→ Minimax
→ α-β pruning

△ → max এর turn

▽ → Min এর turn

Min → Max কে value কে বসায়ু